

DISTRIBUTED MATRIX MULTIPLICATION PROJECT REPORT

Table of Contents

EXECUTIVE SUMMARY	2
SYSTEM OVERVIEW	2
NETWORKING.....	3
REQUEST PROCEDURE CALLS	4
ARCHITECTURES	5
NAMING	6
SYNCHRONIZATION	7
CACHING	9
REPLICATION.....	10
FAULT TOLERANCE.....	11
SCALABILITY ANALYSIS	13
OVERHEAD SUMMARY AND MITIGATIONS	14
CONCLUSION.....	15
REFERENCES.....	15

EXECUTIVE SUMMARY

This report details the design, implementation, and architectural justification of a Distributed Matrix Multiplication System built as part of the Distributed Systems course. The system enables any collection of heterogeneous devices on a Local Area Network (LAN) including Windows, Linux, and macOS computers as well as mobile phones and tablets to collaboratively compute matrix products.

The report documents every concept covered in the course unit: Networking, Remote Procedure Calls, Architectures, Naming, Synchronization, Caching, Replication, and Fault Tolerance. For each concept, the chosen implementation is justified against competing alternatives, the overhead introduced is disclosed, and the mitigation strategy is described.

Primary Reference

- Tanenbaum, A.S. & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms (3rd ed.). All architectural decisions in this system trace directly to principles in this textbook.

SYSTEM OVERVIEW

The Distributed Matrix Multiplication System accepts two matrices of any size from 2×2 up to 200×200 (or beyond) submitted from any device on the network. It partitions Matrix A into row blocks, distributes each block to an available worker node alongside the full Matrix B, collects partial results, and assembles the final product matrix C.

The system supports simultaneous jobs from multiple users, automatic node discovery with zero manual configuration, coordinator fail over without restarting computation, and result persistence for 2 hours after completion.

Component Technology Role

Component	Technology	Role
HTTP REST API	FastAPI + Uvicorn	Job submission, node registration, block assignment
WebSocket	FastAPI WebSocket	Real-time browser worker communication, result delivery
Discovery	mDNS / Zeroconf	Zero-configuration node discovery on LAN
Persistence	SQLite (WAL mode)	All job state, block assignments, results
Browser Compute	JavaScript Web Worker	Mobile phone computation via browser thread
Election	Bully Algorithm	Per-job coordinator failover on node failure

NETWORKING

Network Topology: *Local Area Network over WiFi*

The system operates over a LAN using a standard WiFi router or MiFi hotspot. All devices must be on the same subnet. Internet access is not required at any point during operation. This decision was made deliberately rather than by default. A WAN or VPN-based deployment would introduce variable latency, packet loss, and NAT traversal complexity that would make real-time block assignment and result collection unreliable. Tanenbaum & Van Steen (Section 2.2) identify latency and bandwidth as the two critical network parameters for distributed system design. On a LAN, both are orders of magnitude better than WAN alternatives.

Network Option	Latency	Setup Complexity	Decision
LAN / WiFi (chosen)	< 1ms RTT	Zero - same subnet	CHOSEN
VPN over Internet	20 - 200ms	Certificate management, NAT traversal	Rejected
Ad-hoc WiFi Mesh	Variable	OS-dependent, unstable	Rejected
Ethernet Switch	< 0.1ms	Less portable	Production alternative

Overhead and Mitigation

The MiFi router itself is a single point of failure at the network infrastructure level. This is acknowledged explicitly and distinguished from software-level SPOFs which the system mitigates through replication and election. For production deployment, a managed network switch eliminates this hardware dependency. For the purposes of this prototype, the MiFi provides sufficient reliability.

Communication Protocol: *WebSocket + HTTP REST*

The two protocols serve distinct roles within the system:

- ❖ **HTTP REST** handles infrequent, request-response operations: job submission, node registration, block assignment to Python nodes, heart-beating, and election messages. WebSocket handles persistent, bidirectional connections to browser-based worker nodes (phones and tablets).
- ❖ **gRPC** was the primary alternative considered. It is binary-encoded, type-safe, and significantly faster than HTTP/JSON for Python-to-Python communication. However, browsers cannot speak native gRPC without a proxy layer (Envoy), which would require a running process on every node and contradicts the zero-configuration requirement. Tanenbaum & Van Steen (Section 4.2) note that protocol selection must account for the full range of participating entities, not only the most capable ones.

Protocol	Browser Support	Latency	Binary	Decision
HTTP REST + WebSocket (chosen)	Native	Low (persistent WS)	No (JSON)	CHOSEN
gRPC	Requires Envoy proxy	Very low	Yes (protobuf)	Rejected
Raw TCP sockets	Not supported	Lowest	Yes	Rejected
ZeroMQ	Not supported	Very low	Yes	Rejected

REQUEST PROCEDURE CALLS

Implementation: *HTTP POST as RPC over REST*

The system implements RPC semantics using HTTP POST endpoints. When the coordinator assigns a block to a remote Python worker, it performs an HTTP POST to /work on that worker's FastAPI server. The worker executes the block computation and calls back to the coordinator's /result endpoint with the result. This follows the *at-most-once execution semantics* described in Tanenbaum & Van Steen (Section 4.3) for stateless RPC.

The assignment message is a JSON document containing: *job_id*, *block_id*, *row_start*, *row_end*, *A_block* (the sub-matrix), *B* (the full second matrix), and *attempt_id* (a lease token for stale-result rejection). The result message returns the computed partial *C* matrix and a metrics dictionary containing *compute_time_ms*, *MFLOPS*, *total_operations*, and *device_type*.

RPC vs RMI Decision

Java RMI was considered but rejected because the system must support heterogeneous runtimes (Python, JavaScript). RMI requires both endpoints to run in a JVM. Our RPC implementation is runtime-agnostic: the same JSON message format works identically for Python workers (HTTP) and JavaScript browser workers (WebSocket). Tanenbaum & Van Steen (Section 4.4) identify interface definition language independence as a key advantage of language-neutral RPC over language-specific RMI.

Browser Workers: *WebSocket as RPC Transport*

For mobile phone and tablet browser workers, HTTP POST is not feasible because browser-based workers have no server-side port. The coordinator uses WebSocket instead: it sends an *assign_block* JSON message over the established WebSocket connection. The browser's JavaScript Web Worker thread receives the message, performs the computation, and posts the result back through the same WebSocket channel.

This is functionally identical to HTTP-based RPC but uses the WebSocket connection as the transport layer. The coordinator routes to the correct transport based on the worker's *device_type* field in the node registry. Browser workers have *port=0*, which signals the coordinator to use the WebSocket path rather than HTTP POST.

Idempotency and the attempt_id Lease Token

Tanenbaum & Van Steen (Section 4.3) identify idempotency as a critical property for reliable RPC in the presence of network failures and retries. The system implements a lease token mechanism using *attempt_id*. Each time a block is assigned, the storage layer increments *attempt_id*. Workers return the *attempt_id* alongside their result. The coordinator's *storage.complete_block()* method rejects any result whose *attempt_id* does not match the current value in the database. This prevents stale results from a dead worker's slow computation from overwriting a result from its replacement.

Scenario	attempt_id behaviour	Outcome
Normal completion	Worker returns attempt_id=N, DB has N	Result accepted
Worker times out, block reassigned	DB increments to N+1	attempt_id=N
Slow worker returns late	attempt_id=N != current N+1	Rejected as stale
Coordinator failover, block reset	attempt_id incremented again	New coordinator's result accepted

ARCHITECTURES

Chosen Architecture: *Stateless Dynamic Coordinator (Hybrid)*

The system uses a hybrid architecture combining elements of client-server and peer-to-peer, which Tanenbaum & Van Steen (Section 2.2) term a **hybrid architecture**. The submitting device becomes the coordinator for that specific job. The coordinator assigns blocks to all other nodes as workers. Any node can become a coordinator. No node is permanently designated as master.

The coordinator is explicitly **stateless**: it holds no private in-memory state. All state lives in SQLite. This is the defining architectural decision of the system and is what enables fault-tolerant failover without state transfer protocols.

Architecture	Advantage	Problem for our system	Decision
Pure Master-Worker	Simple, clear authority	Permanent master = SPOF	Rejected
Pure Peer-to-Peer	No SPOF	No decision authority; complex consensus	Rejected
Stateless Dynamic Coordinator (chosen)	Fault-tolerant; any node can lead; simple failover	Requires election protocol	CHOSEN

Stateful Coordinator with Replication	Fast in-memory access	Two sources of truth; sync complexity	Alternative
---------------------------------------	-----------------------	---------------------------------------	-------------

Why Stateless Over Stateful Coordinator

A stateful coordinator keeps job state in memory and replicates it to backup nodes. This is faster (RAM vs disk) but introduces a window of data loss between writes and replication confirmation. It also requires implementing a state transfer protocol for failover.

The stateless design writes every state change to SQLite before acting on it. The overhead is one disk write per state transition (approximately *1-5ms per write* on an SSD). For our workload matrix computation blocks that take *50 - 500ms* to compute: this overhead is negligible. The benefit is that failover reduces to a single database read: the new coordinator reads pending blocks from SQLite and assigns them. No state transfer is needed.

Multiple Simultaneous Jobs

Each job has its own independent coordinator. Five users submitting five jobs simultaneously produce five coordinators, each managing its own subset of workers from the same node pool. Workers can receive assignments from multiple coordinators concurrently. The per-job *job_id* ensures no cross-contamination between concurrent jobs in the database or in result delivery.

NAMING

Node Naming: UUID4 with mDNS Service Announcement

Each node is assigned a **UUID4 identifier** at first startup, persisted to a port-specific file (*.node_id.{port}*). This identifier is stable across restarts and is the authoritative identity of the node within the system. UUID4 provides 122 bits of randomness, making collision probability statistically negligible across any realistic deployment.

Tanenbaum & Van Steen (Section 5.2) classify naming systems by structure: flat, structured, and attribute-based. **UUID4 is a flat naming system**. It provides no location information or hierarchy, which is appropriate because node location in this system is dynamic (IP addresses change when a device rejoins the network).

Service Discovery: mDNS / Zeroconf

The **zeroconf** library implements **mDNS (Multicast DNS)**, which is the protocol used by Apple Bonjour, Chromecast, and network printers for zero-configuration service announcement. Each node registers a service record of the form *{node_id}-{port}._matmul._tcp.local*. When any new node joins the LAN, it announces itself via multicast UDP. All existing nodes on the subnet receive the announcement within approximately 500ms.

Discovery Option	Configuration Required	SPOF	Convergence Time	Decision
mDNS / Zeroconf (chosen)	None	None	~500ms	CHOSEN
Static IP config file	Manual entry	File sync problem	Instant	Rejected
Central Registry Node	Registry address	Registry itself	Instant	Rejected
Gossip Protocol	Bootstrap peer	None	Several seconds	Rejected

Job Naming: UUID4 per Job

Every matrix multiplication job receives a **UUID4** *job_id* at the moment of submission. This identifier is generated by the coordinator before any computation begins and is immediately written to SQLite. It is then communicated to all workers in every block assignment message, to all backup nodes via state sync, and to the submitting client in the HTTP response. Any node on the network can retrieve job state by querying the database with only the *job_id*.

Sequential job IDs were considered as an alternative. They are human-readable and compact but require a central counter, which is itself a SPOF. UUID4 generation is local and requires no coordination. Tanenbaum & Van Steen (Section 5.1) explicitly recommend flat unstructured identifiers for systems where location independence is more important than human readability.

Block Naming: Composite Identifiers

Each block is identified by a composite name: *{job_id}-blk-{index}*. The *job_id* prefix ensures global uniqueness across concurrent jobs. The *-blk-{index}* suffix provides deterministic, human-readable ordering within a job. This naming scheme was chosen over random block UUIDs because the index encodes row ordering information, which is required when assembling the final result matrix from partial results.

SYNCHRONIZATION

Coordinator Election: Bully Algorithm

The Bully Algorithm is used for per-job coordinator election. When a node detects that a job's coordinator has missed three heartbeats (3 seconds), it initiates a **Bully election**. The algorithm proceeds as follows:

1. The detecting node sends an *ELECTION* message to all nodes with a higher *node_id*.
2. If no *OK* response is received within *ELECTION_TIMEOUT* (5 seconds), the node declares victory.
3. If an *OK* is received from a higher node, the detecting node waits for a *COORDINATOR_ANNOUNCE*.
4. The winning node broadcasts *COORDINATOR_ANNOUNCE* to all nodes.

5. The new coordinator reads pending blocks from SQLite and resumes the job.

Elections are scoped to *job_id*. A coordinator failure for Job A does not affect Job B. Multiple elections can run concurrently without interference because each election carries its *job_id* in every message.

Why Bully over Ring Algorithm

The Ring Algorithm requires all nodes to be arranged in a logical ring. Maintaining ring membership under dynamic joins and leaves requires additional protocol machinery. With 40 - 50 nodes, the ***Bully Algorithm's $O(n^2)$*** worst-case message complexity produces at most 2,500 messages, each approximately 200 bytes, totaling 500KB of election traffic on LAN negligible. ***Bully Algorithm is preferred when n is small and membership is dynamic.***

Heartbeat-Based Failure Detection

Every Python node broadcasts a heartbeat message to all known peers every *HEARTBEAT_INTERVAL* (1 second). The heartbeat carries the node's *node_id*, current status (idle, working, coordinating), and a timestamp. A node is considered dead if its *last_seen* time exceeds *HEARTBEAT_TIMEOUT* * 5 (15 seconds). The coordinator health monitor checks every *COORD_HEALTH_INTERVAL* (5 seconds) whether any running job's coordinator has missed sufficient heartbeats.

A 30-second startup grace period (*STARTUP_GRACE*) prevents a newly joined node from immediately triggering elections for all running jobs it finds in the database before it has received any heartbeats from their coordinators. This is a critical correctness property: without it, a new node would declare every existing coordinator dead and trigger concurrent spurious elections.

Block Timeout and Reassignment

If a worker receives a block but fails to return a result within *BLOCK_TIMEOUT* (30 seconds), the coordinator's *_monitor_until_done* loop detects this through *get_timed_out_blocks()* and reassigns the block to itself. The block's *attempt_id* is incremented on reassignment, so any late result from the original worker is automatically rejected by the stale-result protection in *storage.complete_block()*.

Synchronization Event	Mechanism	Timeout / Period	Recovery Action
Node failure detection	Heartbeat monitoring	15s without heartbeat	Prune from registry
Coordinator failure	Health monitor check	5s check interval	Bully election
Block worker timeout	<i>get_timed_out_blocks()</i>	30s without result	Reset and reassign to self
Startup grace period	<i>uptime < 30s</i> check	30s from startup	Skip health checks
Stale result rejection	<i>attempt_id</i> comparison	On every result receipt	Silent discard

CACHING

Result Caching: SQLite with 2 - Hour TTL

Completed job results are cached in the SQLite results table with a *two-hour time-to-live (TTL)*. The *expires_at* column stores the Unix timestamp at which the result becomes invalid. A background cleanup task runs every 30 minutes to evict expired records. When a client requests a result (*GET / jobs/{job_id}*), the server checks *expires_at* before returning data. Expired results return null rather than stale data.

Tanenbaum & Van Steen (Section 7.1) define caching as storing copies of data close to clients to reduce access latency. In our system, the SQLite database on each node serves as a local cache of job state and results. A client that disconnects and reconnects does not need to recompute; it retrieves the cached result from the database.

Matrix B Broadcast as an Implicit Cache

The row-block partitioning strategy sends the full Matrix B to every worker. Rather than workers fetching B on demand from the coordinator, B is pushed to them at assignment time and held in worker memory for the duration of the computation. This is a push-based implicit caching strategy. The worker's local memory acts as a cache for B, eliminating any repeated fetches during the triple-loop computation.

Cache Coherence Note

Because B is read-only during computation, cache coherence is trivially maintained. No worker ever writes to B. This is a key advantage of the row-block partitioning strategy over block partitioning schemes that require partial B updates. Tanenbaum & Van Steen (Section 7.5) note that *read-only data requires no coherence protocol*.

SQLite WAL Mode as a Write-Through Cache

SQLite is configured in **Write-Ahead Logging (WAL)** mode. WAL mode allows concurrent readers while a writer is active, because writes go to a separate WAL file and readers access the original database file. This is functionally equivalent to a write-through cache: reads see consistent data from the main database file even while writes are in progress on the WAL file. Checkpointing merges the WAL back into the main database file periodically.

Cache Type	Location	TTL	Eviction Policy
Result cache	SQLite results table	2 hours	TTL-based, background cleanup every 30min
Job state cache	SQLite jobs + blocks tables	2 hours (with job)	Deleted with parent job
Matrix B (implicit)	Worker process memory	Duration of computation	Released on block completion
Node registry	In-memory _registry dict	15s heartbeat timeout	Pruned on missed heartbeats

REPLICATION

SQLite State Replication to Backup Nodes

Every state change made by the coordinator is replicated to three backup nodes ($NUM_BACKUP_NODES = 3$) via HTTP POST to `/sync/state`. The payload is a `state_sync` message containing the operation name and the data required to reproduce that operation. Backup nodes apply the operation to their own local SQLite database using `storage.apply_sync()`.

This is an active replication strategy: all replicas apply the same operations in the same order. Tanenbaum & Van Steen (Section 7.4) distinguish between primary-based protocols (where all writes go to a primary) and replicated-write protocols (where all replicas apply writes). Our approach is primary-based with active replication: the coordinator is the primary, and replicas receive and apply the same operations.

Backup Node Selection

At job submission time, the coordinator selects the three Python nodes (not browser nodes) with the oldest `join_time` as backup nodes. Oldest `join_time` correlates with stability: a node that has been on the network for the longest period is statistically least likely to disconnect. Browser nodes are explicitly excluded from backup selection because they have no persistent storage and may disconnect at any time.

Replication Operations

The following state transitions are replicated synchronously before the coordinator proceeds:

- **create_job** - written to backups before any blocks are assigned
- **create_block** - written to backups for each block before assignment begins
- **assign_block** - written after each block assignment
- **complete_block** - written after each block result is accepted (includes `attempt_id`)
- **fail_block** - written when a block is reset for reassignment
- **update_job_status** - written on RUNNING and COMPLETE transitions
- **store_result** - the completion flag is synced; the full matrix is not re-sent to save bandwidth

Replication Overhead and Mitigation

Synchronous replication to three nodes adds latency to every state transition. On LAN, each HTTP POST takes approximately 1–3ms. Three replicas add 3–9ms of overhead per state change. For a 100×100 job split into 50 blocks, this is **50 assign + 50 complete = 100** replication rounds, totaling approximately **300 - 900ms of aggregate replication overhead**.

This is mitigated in two ways: first, `asyncio.gather()` sends replication requests to all three backups concurrently rather than sequentially (parallel overhead instead of serial). Second, failed replications are retried up to `SYNC_RETRY_COUNT` (2) times with 0.5s backoff. If a backup remains unreachable after retries, it is removed from the active backup list for that job. The job proceeds with fewer backups rather than blocking.

Why Not Asynchronous Replication

Asynchronous replication would reduce per-operation latency but introduces a window during which a coordinator failure could result in data loss. Tanenbaum & Van Steen (Section 7.4) note that synchronous replication is appropriate when consistency guarantees outweigh throughput concerns. For a system where recomputing a block wastes CPU time across multiple devices, the consistency guarantee is worth the latency cost.

FAULT TOLERANCE

Failure Model

The system assumes a crash-stop failure model for individual nodes: nodes fail by stopping, not by sending incorrect messages. This is appropriate for a trusted LAN where all nodes run the same codebase. Byzantine failures (nodes sending deliberately incorrect results) are not addressed in this implementation, consistent with the system's explicit trust assumption.

Tanenbaum & Van Steen (Section 8.2) classify failure modes as crash failures, omission failures, timing failures, and Byzantine failures. The system is designed to tolerate crash failures and mission failures (dropped messages). Timing failures are handled through heartbeat timeouts and block reassignment.

Coordinator Failover: Mid-Computation Recovery

The most critical fault tolerance property is mid-computation recovery without restarting. When the coordinator crashes:

1. The health monitor on surviving nodes detects the coordinator's absence (3 missed heartbeats over 3 seconds, confirmed at the next 5-second health check).
2. A Bully Election runs among surviving nodes (~1 second to complete).
3. The winning node calls *storage.update_job_coordinator()* to claim ownership of the job.
4. It calls *storage.get_pending_blocks()* and *storage.get_timed_out_blocks()* to determine which
5. blocks still need computation.
6. Timed-out blocks are reset to pending via *storage.fail_block()*.
7. The sequential *_run_job* pipeline resumes with only the pending blocks.
8. Already completed blocks are never recomputed.

Total recovery time: approximately 3–5 seconds from coordinator failure to new coordinator beginning block assignment. This is the acknowledged cost of the fault tolerance mechanism.

Result Delivery to Reconnected Clients

If the submitting client disconnects before receiving the result, the result persists in SQLite for 2 hours. When the client reconnects via WebSocket, the server checks for pending results and pushes them immediately. This is implemented in the WebSocket handler: on *browser_register*, the server checks *localStorage* for the last submitted *job_id* and calls *pollResult()*.

Stale Result Protection

The *attempt_id* lease token in *storage.complete_block()* prevents stale results from corrupting the accumulated partial matrix. When a block times out and is reassigned, the *attempt_id* is incremented. Any result from the original (now-assumed-dead) worker arrives with the old *attempt_id*, which no longer matches the current value in the database, and is silently rejected. ***This is a direct implementation of the idempotent receiver pattern*** described in Tanenbaum & Van Steen (Section 4.3).

Browser Worker Disconnection Tolerance

Browser workers (phones and tablets) can disconnect at any time without notice. When a WebSocket disconnects, the WebSocket handler removes the browser node from the *_browser_sockets* registry and calls *storage.remove_node()*. Any blocks assigned to that browser node will time out within *BLOCK_TIMEOUT* (30 seconds) and be reassigned by the coordinator's monitoring loop. The block's *attempt_id* increment ensures the disconnected browser's eventual result (if it reconnects and completes late) is rejected.

Failure Type	Detection Method	Detection Time	Recovery Action	Recovery Time
Coordinator crash	Heartbeat timeout + health monitor	3–8s	Bully election + DB resume	4–5s
Worker crash (Python)	Block timeout	30s	Reset block, reassign to self	< 1s after detection
Browser disconnect	WebSocket close event	Immediate	Remove from registry; timeout reassigns block	30s max
Result arrives stale	<i>attempt_id</i> mismatch	Immediate (on receipt)	Silent reject; correct result accepted	0s
Backup node unreachable	HTTP POST failure after retries	~1.5s	Remove from backup list; continue	< 2s

SCALABILITY ANALYSIS

Computational Scalability

For matrices A ($m \times k$) and B ($k \times n$), the total computation requires $2 \cdot m \cdot k \cdot n$ floating-point operations (one multiply and one add per inner product step). With W workers each handling m/W rows:

- Work per worker: $(m/W) \cdot k \cdot n$ operations
- Theoretical speedup: $S(W) = W$ (embarrassingly parallel - workers never communicate)
- Amdahl's Law: $S = 1 / (1 - P)$, where $P \approx 0.99$ (only assembly is serial)

Because the row-block strategy is embarrassingly parallel (Tanenbaum & Van Steen, Section 11.3), speedup scales linearly with worker count up to the number of rows in Matrix A. For a 100×100 matrix with 100 workers, each handles 1 row independently with zero inter-worker communication.

Numerical Scalability Examples

Matrix Size	Workers	Block Size	Theoretical Speedup	Network Cost (B broadcast)
10×10	5	2 rows each	5×	$5 \times 5\text{KB} = 25\text{KB}$
100×100	50	2 rows each	50×	$50 \times 80\text{KB} = 4\text{MB}$
200×200	50	4 rows each	50×	$50 \times 320\text{KB} = 16\text{MB}$
500×500	100	5 rows each	100×	$100 \times 2\text{MB} = 200\text{MB}$
1000×1000 (theoretical)	500	2 rows each	500×	$500 \times 8\text{MB} = 4\text{GB}$ - requires SUMMA

Scalability to 500,000 - 1,000,000 Concurrent Users

The system is validated experimentally with 3 - 25 devices. The following analytical argument demonstrates scalability to 500,000 - 1,000,000 concurrent users:

Each concurrent user submits one job. Jobs are independent: they do not share state except the shared pool of available workers. The bottleneck is therefore not the distributed system architecture but the total available compute. With N total worker nodes and J concurrent jobs, each job receives N/J workers on average.

- For $J = 1,000,000$ concurrent jobs and $N = 1,000,000$ worker nodes: each job receives 1 worker on average. Each worker is a full node executing the entire job sequentially. This degrades gracefully to single-node performance rather than failing.
- For $J = 1,000,000$ jobs and $N = 10,000,000$ workers: each job receives 10 workers, achieving $10\times$ speedup per job. The system scales horizontally by adding worker nodes.

The M/M/k queuing model applies for job throughput analysis. With k workers and exponential service time, the utilisation $\rho = \lambda/(k\mu)$ must be less than 1 for stability. With arrival rate $\lambda = 1,000$ jobs/second, average service time $1/\mu = 1$ second per block, and $k = 50,000$ workers: $\rho = 1000/(50000) = 0.02$. **The system operates at 2% utilization:** well within stable bounds.

Production scaling path

For $n > 100$ nodes, the B-broadcast strategy becomes a network bottleneck. The production scaling path is to replace B broadcast with **SUMMA** (Scalable Universal Matrix Multiplication Algorithm), which column-partitions B across nodes and uses a ring communication pattern. This reduces per-node network cost from $O(k \cdot n)$ to $O(k \cdot n/W)$, achieving near-linear scaling to thousands of nodes. This is documented in the design document but not implemented in the prototype.

OVERHEAD SUMMARY AND MITIGATIONS

Concept	Overhead Introduced	Magnitude	Mitigation
Networking	MiFi as hardware SPOF	Total outage if MiFi fails	Replace with managed switch in production
RPC (HTTP)	JSON serialisation latency	~0.1–0.5ms per call	Acceptable at LAN scale; MessagePack as future optimisation
RPC (attempt_id)	One DB write per assignment	~1–5ms per block	Negligible vs block compute time (50–500ms)
Architecture (election)	3–5s compute pause on failover	Linear in election frequency	Pre-elect standby coordinator (future work)
Naming (mDNS)	Multicast UDP traffic	~300 bytes per announcement	Negligible; 50 nodes $\approx$ 15KB total
Naming (port-specific ID file)	Additional file per port	Bytes on disk	None required; trivial
Synchronization (heartbeat)	1 UDP/HTTP message/second/node	50 nodes $\approx$ 50 msg/s	Negligible on LAN
Synchronization (grace period)	30s delay before health checks	One-time per node startup	Prevents spurious elections; accepted
Caching (SQLite WAL)	Checkpoint I/O every ~1000 writes	Periodic disk flush	SYNCHRONOUS=NORMAL reduces fsync frequency

Replication (3 backups)	3 parallel HTTP POSTs per state change	3–9ms per operation (parallel)	Parallel with <code>asyncio.gather()</code> ; retry on failure
Fault Tolerance (B broadcast)	Full B sent to every worker	$80\text{KB} \times 50 = 4\text{MB}$ per 100×100 job	Acceptable to 100 nodes; SUMMA required beyond

CONCLUSION

The Distributed Matrix Multiplication System demonstrates the application of all core distributed systems concepts in a single coherent implementation. Every architectural choice: mDNS for naming, the Bully Algorithm for synchronization, SQLite with WAL for caching and replication, the stateless coordinator model for architecture, HTTP REST and WebSocket for RPC, and the *attempt_id* lease token for fault tolerance: is justified against its alternatives with explicit overhead disclosure.

The system operates correctly with 3-25 devices tested experimentally. The analytical arguments in **Section 11** demonstrate that the architecture scales to 500,000 - 1,000,000 concurrent users through horizontal worker addition, with graceful degradation under overload rather than catastrophic failure. The B-broadcast bottleneck at $n > 100$ nodes is the identified production scaling boundary and the documented next engineering milestone.

The *stateless coordinator* design is the most significant departure from classical distributed systems textbook examples, which typically assume stateful masters with expensive failover protocols. By making the coordinator entirely dependent on SQLite, failover reduces to a database read followed by block reassignment - a solution that is simultaneously simpler, more reliable, and more verifiable than any in-memory state transfer approach considered.

REFERENCES

- Tanenbaum, A.S. & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms (3rd ed.). Pearson. - **Primary reference for all architectural decisions.**
- Garcia-Molina, H. (1982). Elections in a Distributed Computing System. IEEE Transactions on Computers, 31(1), 48–59. - **Original Bully Algorithm paper.**
- SQLite Consortium. (2024). Write-Ahead Logging. <https://www.sqlite.org/wal.html>
- FastAPI Documentation. <https://fastapi.tiangolo.com>
- Python Zeroconf. <https://python-zeroconf.readthedocs.io>
- MDN Web Docs. Web Workers API. https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API
- van Renesse, R., et al. (1996). A gossip-style failure detection service. Middleware '96. - **Referenced for heartbeat failure detection design.**